

Building the DLP Agent: Trust, Licensing, and Data

Authentication and Authorization, the Subscription Model, and Data Storage — a practical guide written for engineers.

Project: On-Premise Endpoint Data Leak Prevention (Windows)

Audience: software engineers joining the project (no prior DLP background assumed)

Status: internal engineering design guide

How to read this guide

This is a teaching document, not a spec. Each section starts from the idea, uses a plain analogy, then gets concrete about what you build and why. If a term is new, check the Glossary at the end. The four things we cover — how the parts prove who they are (authentication), what each is allowed to do (authorization), how we license and charge for the product (subscription model), and how we keep captured data safe (storage) — are the load-bearing skeleton of the whole system. Get these right and the detection features hang off them cleanly.

0 The product in one page (so the rest makes sense)

We are building software that stops sensitive data — classified documents, decisions, plans — from leaving an organisation through its employees' Windows PCs. There are two big pieces you need to picture before anything else.

- **The agent.** A small program that runs on every PC. It watches the ways data can leave (USB drives, email, web uploads, cloud sync, printing, clipboard, and so on), inspects the *content* of what is about to leave, and blocks it if it is sensitive.
- **The management server.** One machine per site, living inside the customer's own building. It is the brain: it holds the rules (policies), the licence, every recorded incident, and the dashboard the security team looks at.

The single most important product rule: **everything stays on the customer's premises**. The vendor (us) never receives the customer's data. This is called *data residency*, and it is why these buyers — defence, government, nuclear and power plants — can use us at all. Keep this rule in your head; it shapes every decision below, especially licensing and storage.

A mental model to carry through the whole document

Think of the management server as a **head office** and each agent as a **field officer** posted at a PC. The field officers need ID cards that the head office issues and checks (authentication and authorization). The head office needs to know how many officers it is **allowed** to field, and that permit must be genuine (licensing). And every case file the officers send back must go into a **secure cabinet** (data storage). The rest of this guide is just those three ideas, in detail.

1 First principles: authentication vs authorization

These two words get mixed up constantly, so pin them down now. They answer two different questions:

- **Authentication (authN)** — “Are you really who or what you claim to be?” It is about *proving identity or genuineness*.
- **Authorization (authZ)** — “Now that we know who you are, what are you allowed to do?” It is about *permissions and scope*.

The airport analogy (remember this one)

At the airport, showing your passport at the desk is **authentication** — you are proving you are really you. Your boarding pass is **authorization** — it says which flight and seat you may board. Notice they are separate: you can be perfectly authenticated (it really is you) and still be *unauthorized* to walk into the cockpit. Good systems check identity first, then check permission for the specific action — and never let one silently stand in for the other.

The practical rule for your code: **authenticate once to establish identity, then authorize every sensitive action against that identity**. A common junior mistake is to assume that because a caller got past the login, they are allowed to do whatever they ask. They are not. Identity and permission are two gates, not one.

2 The three planes of trust in this system

Our system has three very different kinds of “actor” that each need their own authentication and authorization. Trying to use one mechanism for all three is a mistake. The three planes are: the **licence** (a thing we issue), the **agents** (software on PCs), and the **admins** (the humans using the console).

2.1 Plane 1 — the licence: is it genuine, and what does it permit?

The licence is not just a key string you type in. It is a small block of data that has been **cryptographically signed**. The data inside says: which customer, which site, how many seats (say 1000), which optional modules are on, and an expiry date. We sign that block with a **private key** that lives only in a hardware security module (HSM) in our back office. Every management server we ship has the matching **public key** built into it.

Authentication of the licence

The server verifies the signature using that embedded public key. If someone edits the seat count or forges a licence, the signature no longer matches and it is rejected. If the customer bought the hardware-dongle option, the **dongle itself is the authenticator**: it is a tamper-resistant crypto token whose secret keys never leave it; the software asks it challenge questions and it returns cryptographic proof that it is genuine and physically present. There is also a **node-lock** check: the server re-computes a fingerprint of its own hardware on every start and confirms it matches the licence, so copying the licence file to a second machine fails.

Authorization from the licence

Once the licence is proven genuine, the server reads the fields and *enforces* them: allow up to N agents, unlock module X, keep working until date Y. It counts how many agents are actually live and refuses to hand out entitlement beyond N. That counting-and-limiting is authorization.

In one line

Checking the licence’s signature (or the dongle) is **authentication** — is this permit real? Reading “valid for 1000 seats, fingerprinting module included, expires Dec 2026” and enforcing it is **authorization** — what does this real permit allow?

2.2 Plane 2 — agent and server: is this agent legitimate, and what may it do?

Here is the hard part. A thousand agents must trust the server, and the server must trust each agent — with no internet and no outside certificate authority to lean on. The solution is that **the server runs its own private certificate authority (CA)** when you first set it up. Everything is issued and checked locally.

How an agent joins (enrollment)

1. The agent installer carries a **one-time enrollment token** — a bootstrap secret baked into the install configuration.
2. On first run, the agent contacts the server and presents that token.
3. The server validates the token and issues the agent its own **unique client certificate**, signed by the server’s internal CA.

From then on, every check-in uses **mutual TLS**: the server proves its identity to the agent (server certificate) *and* the agent proves its identity to the server (client certificate). A random PC cannot join without a valid token, and a fake or impersonating server cannot feed commands to the agent because it cannot present a certificate the agent trusts. This is also what stops an attacker from extracting the agent

binary and pointing it at a counterfeit console. All of that is **authentication**.

What the agent is allowed to do (authorization)

After the agent authenticates, the server draws one seat from the licence pool and issues the agent a short-lived, **signed entitlement token** that authorizes it to run in enforcing mode, plus the **signed policy bundle** it is permitted to apply. The agent checks the server's signature on both, so even an attacker on the same network cannot inject fake policy. The token has a short lifetime and must be renewed at each check-in — so if the agent is removed from the deployment, or the licence lapses or is over-subscribed, the token simply is not renewed and that agent falls out of entitlement.

Analogy

The internal CA is the head office's own **badge printer**. Certificates are the ID badges. Mutual TLS is both people showing their badge at every meeting. The entitlement token is **today's signed duty roster** telling the officer they are on active duty — and it expires, so it has to be re-issued regularly.

2.3 Plane 3 — admin and console: is this person who they claim, and what may they touch?

Authentication: admins sign in to the on-premises console against the customer's own directory (Active Directory / LDAP), or local accounts on fully isolated sites, ideally with multi-factor authentication.

Authorization: role-based access control (RBAC) with *separation of duties*. There are distinct roles — policy author, incident reviewer, auditor, system administrator — so the person who writes the rules is not automatically the person who reviews who broke them or who can export the evidence. Every admin action is written to a tamper-evident audit log.

Why separation of duties matters

If one super-admin could write policy, review incidents, export the sensitive evidence, and delete the audit trail, then a single malicious insider could both leak data and erase the proof. Splitting these powers across roles means no one person can do harm unnoticed. Accreditors specifically look for this.

2.4 The golden rule that ties Plane 1–3 together

Everything verifies locally — nothing phones home at runtime

Licence authenticity is a signature check against a key we embedded. Agent identity is the customer's own internal CA (no external certificate look-ups). Admin identity is the customer's own directory. None of it needs to reach the internet while running. **This is exactly what lets the product work in a firewalled or air-gapped site, and it is a major selling point to defence buyers.** Whenever you add a feature, ask: "does this still work with the network cable unplugged from the outside world?" If not, rethink it.

3 The licensing and subscription model, and how to build it

3.1 What we are actually selling

The natural unit is **per-endpoint (per-seat)**: the customer buys the right to protect a number of PCs. We sell in bands with a minimum floor. On top of that unit sit three commercial shapes, and we offer more than one because different buyers budget differently.

Commercial model	Who it suits	How we bill
Perpetual + annual maintenance (AMC)	Government and defence buyers whose tenders and capital budgets favour a one-time purchase.	One-time licence fee, then ~18–22% per year for support and updates.
Term subscription (1 / 3 / 5 year)	Buyers who prefer operating-expense budgets and predictable renewals.	A recurring fee for a fixed term; stops working (see fail-secure below) if not renewed.
Per-site flat licence	Very large or “keep it simple” sites that do not want to count seats.	A single fee per site, capped or uncapped, regardless of exact PC count.

Optional modules (device control, content-aware fingerprinting, OCR, and so on) are unlocked by the same licence, so we can upsell a customer more capability *without reinstalling anything* — we just issue an updated licence.

3.2 How enforcement actually works (the key design)

Do **not** node-lock every single agent — activating a thousand machines individually is operationally miserable. Instead:

- The **server holds the master entitlement** (N seats, which modules, expiry), bound to a root of trust: either a **hardware dongle** (an EAL5+ smart-card token that works fully offline and whose secure internal clock enforces subscription terms) or a **software node-lock** (a licence file bound to a fingerprint of the server — convenient for virtual machines, but the fingerprint can drift on snapshot or migration, so you need a clean re-host process).
- Agents **draw entitlement from the server’s pool**. The server meters live agents against N. Copy an agent binary to an unlicensed machine and it never receives a valid entitlement token, so it will not run in enforcing mode.

Licensing doubles as anti-cloning

Because a copied agent cannot get a valid, server-signed entitlement token, the licence system also helps stop someone cloning your product. The licence controls both “how many” and “is this even a legitimate deployment.” It works together with the code-obfuscation and dongle-gated decryption we use to protect the binaries.

3.3 The licence lifecycle

1. **Order.** The customer buys N seats, chosen modules, and a term.
2. **Generate and sign.** Our back office creates the entitlement block and signs it with the private key inside the HSM.
3. **Deliver.** We either program and ship a dongle, or issue a signed licence file / activation code.
4. **Activate on the server.** Online (one call to our licence service) or, for isolated sites, an offline file exchange (next section).
5. **Distribute to agents.** The server issues signed per-agent entitlement tokens and meters the count.
6. **Renew.** Online renewals happen automatically at check-in; offline sites import a new signed update file each period; a dongle’s stored expiry is updated by an offline update.
7. **Expand or true-up.** To add seats we issue an update that raises N; the console reports the real deployed count for audit.

8. **Revoke or transfer.** To move to new hardware, the old licence is returned or re-hosted with a return file; compromised dongles can be blocked with a blacklist update.

3.4 Online vs offline activation (why the “file round-trip” matters)

Many of our sites are firewalled or air-gapped, so activation cannot assume internet. The answer is a **context-file round-trip**, and it is the same idea as offline activation on other licensed software:

1. Server exports a REQUEST file (contains the machine fingerprint; no secrets) e.g. CodeMeter *.WibuCmRaC / C2V
2. Customer sends it to us (portal, email, or removable media)
3. We generate a signed UPDATE file (the actual entitlement)
4. Customer imports it on the server -> licence is now active

No live connection to us is ever required. The request file carries only a hardware fingerprint, never a secret, so it is safe to move across an air gap on a USB stick.

3.5 What happens when a licence lapses (an important decision)

Fail secure, not fail open

For a **security** product, disabling protection because a licence lapsed would be dangerous — data could walk out the door simply because someone forgot to renew the AMC. So we **fail secure**: agents keep enforcing the existing policy, while the console warns loudly and then progressively restricts *administrative* actions (new enrollments, policy edits, updates) rather than switching off protection. The same principle applies if the server is temporarily unreachable: agents enforce their cached policy and reconcile when they reconnect.

3.6 How to actually build the licensing subsystem

Buy the hard crypto, do not roll your own

Use a proven licensing SDK (for example Wibu CodeMeter or Thales Sentinel) rather than inventing your own licence cryptography, dongles, node-locking, and offline activation. Home-grown licence crypto is a classic trap — it is easy to get subtly, fatally wrong. Let the SDK handle signing, tamper resistance, and the file round-trip; you build the business logic around it.

Around that SDK, these are the components you will build:

- **Back-office licence generator + HSM.** Creates and signs entitlements. This is the *only* place the private signing key ever exists.
- **Licence service / portal.** Handles online activation and the offline file exchange; issues renewal and expansion update files.
- **Activation module on the server.** Talks to the dongle or SDK, verifies the licence, and stores the active entitlement.
- **Entitlement issuer.** Draws seats from the pool and signs the short-lived per-agent tokens.
- **Metering and compliance reporting.** Counts live agents, flags over-subscription, and feeds the dashboard.

Security rules for this subsystem: the private signing key never leaves the HSM and is never shipped; treat the enrollment token like a password (never hard-code it in source or print it to logs); and write every licence event to the audit log.

4 Data storage: what we keep, where, and how we protect it

4.1 What we store

- **Incident metadata** — who, which PC, which channel, which detector fired, severity, action taken, timestamp.
- **Evidence** — the offending file or fragment and its context. This can itself be classified (it is literally the thing someone tried to leak).
- **Policies** and their version history.
- **Endpoint inventory and agent health / state.**
- **Fingerprint index** (the EDM / IDM hashes). These are irreversible — you cannot reconstruct the original data from them.
- **Audit logs** of admin actions — append-only.
- **Users and roles.**

4.2 Where it lives

Split storage by shape and sensitivity. Put the structured records in a relational database (**PostgreSQL** is a solid default): incident metadata, policies, inventory, users and roles, and audit. Put the **evidence blobs in a separate, encrypted store** (filesystem or object store) rather than jamming them into the database — this keeps the database light and lets you protect the sensitive evidence with its own, stronger controls. The fingerprint index lives on the server and is also pushed (size-capped) to endpoints so matching works even when a PC is offline.

4.3 Encryption at rest and key management

Encrypt evidence with **AES-256**. Hold the keys in a **FIPS 140-3 validated module or HSM**, not in a config file. Consider a **per-incident key** so that compromising one record does not expose all of them. Use defence in depth: full-disk encryption, database-level encryption (TDE), and per-blob encryption are *different layers* — do not rely on just one.

4.4 Handling classified evidence (the tricky bit)

Sometimes the safest thing to store is nothing

The evidence you capture may be TOP SECRET. Offer two modes as a per-classification policy setting: (a) store the full fragment, encrypted, for forensic detail; or (b) for the highest classifications, store **only a hash and metadata** — enough to prove a match happened and record the incident, without keeping a second copy of the secret. Access to the evidence store is always RBAC-gated, and every single access is audited.

4.5 Retention and secure deletion

Make retention configurable (for example 90 / 180 / 365 days) to match the customer's compliance rules. For deletion, prefer **crypto-shredding**: if each record is encrypted with its own key, then destroying that key makes the data unrecoverable instantly — even in old backups — which is far cleaner and more provable than trying to wipe bytes.

4.6 A starter data model

You do not need this to be perfect on day one, but here is a sane set of core tables to begin from. Note the “Sensitivity” column — it tells you which tables need the heavier encryption and access controls.

Table	Purpose	Key fields	Sensitivity
endpoints	Every managed PC	id, hostname, user, os, last_seen, status	Low
agents	Agent identity and enrollment	endpoint_id, cert_serial, enrolled_at, version	Medium
policies / policy_versions	Rules and their history	id, name, body, version, signed_by	Medium
incidents	Every block / alert	id, endpoint_id, channel, detector, severity, action, ts	Medium
evidence_blobs	The captured content	incident_id, blob_ref, key_id, hash, mode	High
fingerprint_sets	EDM / IDM hash indexes	id, type, hash_data, salt_ref	Medium
audit_log	Admin actions (append-only)	id, actor, action, target, ts, prev_hash	High
admin_users / roles	Console accounts and RBAC	id, principal, role, mfa_enrolled	Medium
license_state	Active entitlement	seats, modules, expiry, bound_fingerprint	Medium
agent_entitlements	Per-agent live tokens	agent_id, token_hash, issued_at, expires_at	Medium

5 How to move forward: a build order for these subsystems

When you start coding, build the **skeleton** before the clever detection. Auth and storage are what everything else hangs on, so do them first and in this order:

1. **Foundation.** Stand up the server, create the PostgreSQL schema (endpoints, policies, incidents, audit), add admin login (local accounts first, directory integration later), and wire up RBAC.
2. **Secure channel (Plane 2).** Build the internal CA, the enrollment handshake (token, then certificate), mutual TLS, and a basic agent check-in. Goal: get *one* agent talking to the server securely.
3. **Licensing.** Integrate the licensing SDK, add activation (online first, offline round-trip next), seat metering, per-agent entitlement tokens, and the fail-secure behaviour.
4. **Evidence storage.** Add the encrypted blob store, AES-256 with proper key management, retention, and the metadata-only option for the highest classifications.
5. **Dashboard and audit.** Build the views over the data (incidents, channels, per-PC drill-down, licence consumption, agent health), the audit-log view, and reporting exports.
6. **Harden.** Move keys into an HSM, adopt a FIPS-validated crypto module, build the offline update pipeline, add tamper-attempt alerts, and get the whole thing penetration-tested ahead of accreditation.

The one habit to build

For every feature, keep asking two questions: “Does this still work with no internet?” and “Is the sensitive data encrypted and access-audited at every step?” If the answer to either is no, you are not done yet.

6 Common pitfalls to avoid

Mistakes that are easy to make and expensive to fix later:

- Rolling your own licence cryptography instead of using a proven SDK.

- Conflating authentication and authorization in code — treat them as two separate gates.
- Failing *open* on a licence lapse. For a security product, keep protecting; restrict admin instead.
- Storing evidence unencrypted, or in the same table as its metadata.
- Hard-coding the enrollment token in source or writing it to logs. It is a secret.
- Assuming internet access. Design activation, updates, and certificate checks to work fully offline.
- Letting the private signing key leave the HSM, or shipping it anywhere. It never leaves.
- Skipping the audit log. You need it for both forensics and accreditation.
- Pushing agents or kernel drivers to all machines at once. Roll out in staged rings.
- Forgetting that virtual-machine fingerprints drift — have a clean licence re-host process ready.

7 Glossary

Term	Plain-language meaning
Authentication (authN)	Proving something is genuine / who it claims to be.
Authorization (authZ)	Deciding what an authenticated party is allowed to do.
Certificate Authority (CA)	A trusted issuer of digital ID certificates. Ours is internal and runs on the server.
Mutual TLS (mTLS)	An encrypted connection where both sides prove their identity with certificates.
HSM	Hardware Security Module — a tamper-resistant device that stores keys and signs data; keys never leave it.
Dongle	A hardware licence token (USB smart card). Proves the licence is present and genuine; works offline.
Node-lock	Binding a licence to one machine's hardware fingerprint so it cannot be copied elsewhere.
Entitlement token	A short-lived, signed permission the server gives an agent to run in enforcing mode.
EDM / IDM	Exact Data Matching / Indexed Document Matching — fingerprinting the customer's own data as irreversible hashes.
RBAC	Role-Based Access Control — permissions granted by job role rather than to individuals.
Separation of duties	Splitting powerful actions across roles so no single person can act unchecked.
Fail secure	On failure, stay protective (keep blocking) rather than opening up (fail open).
AMC	Annual Maintenance Contract — the yearly fee for support and updates on a perpetual licence.
Crypto-shredding	Deleting data by destroying its encryption key, making it instantly unrecoverable.
TDE	Transparent Data Encryption — database-level encryption at rest.
Air gap	A network with no connection to the outside internet.
Data residency	The guarantee that data stays within a defined location — here, on the customer's premises.

End of guide — keep this handy while you build. When in doubt, favour the choice that keeps sensitive data on-premises, encrypted, and audited.